

ASearcher (rung 3): data preparation, context-length setup, and training infrastructure

CA-Benchmark project — 4B feasibility campaign (P1/P2 + two-stage data filter)

2026-07-28

Scope. What was done to turn the raw ASearcher QA data into RL-ready training data for Qwen3-4B-Instruct-2507 with a single `<search>` tool over local wiki-18 RAG, how the 16k/8-turn context budget was chosen and verified, and the worker/retriever infrastructure that runs it (including the reliability fixes hard-won during the pass-rate diagnostic). All scripts live under `scripts/asearcher/` on branch `agentic-rl-ca`; commits are referenced inline. Companion reports: dataset inspection (`docs/reports/2026-07-25_asearcher_audit`), ScienceWorld audit (2026-07-23).

1 Raw data and base conversion

Sources. inclusionAI/ASearcher-train-data provides QA pairs only (no trajectories): *Base-35k* (35,583 rows: `question`, `answer`, `aug_answer`, `qid`, `source`) and *LRM-35k* (35,054 rows: `question`, `answer`, `id`, `idx`, `source` present on only 11,028 rows; no `aug_answer`). Raw JSONL is staged at `hdfs:.../agentic_rl_ca/data_asearcher_raw/{base,lr}`

split	source slice	rows	note
base	compose_chain_qa	8,537	
base	opensource-fm8k / opensource	7,991 / 7,927	
base	compose_group_qa	6,375	
base	compose_chain_qa_invalid	4,753	dropped (13.4%, user decision 2026-07-27)
lrm	(no source key)	24,026	chain-QA style
lrm	chain_qa_fuzzy / math / webwalker_hard	3,527 / 2,930 / 2,572	webwalker = eval-leakage risk

Conversion (`convert_base_to_parquet.py`, commit `6d6aff8`): emits the `verl/Search-R1` parquet schema (system + user prompt; `reward_model.ground_truth.target`; `tools_kwargs/env_kwargs` carrying question + ground truth). The EM/sub-EM target is the *deduped union of* `answer` + `aug_answer` aliases, so any alias counts as correct (LRM has no aliases; the single answer string is used). Empty-target rows are dropped (zero-gradient otherwise). A 512-row validation split is carved (seed 0). First-pass output (pre-filter): `data_asearcher_base/` = 30,318 train + 512 val — this is what P1/P2 trained on.

2 Contamination finding and the two-stage filter

Why P2 was killed. A P1 trajectory dump (384 trajs) showed **305/384 rollouts issued zero searches** and **87/114 correct answers came from parametric memory**. The stage-1 (closed-book) filter that ASearcher’s recipe presumes had been skipped; P2’s rising val EM was measuring recall, not search. Both P2 arms were killed at ~step 60/30 and the data pipeline rebuilt.

Stage 1 — closed-book filter (`closed_book_filter.py` + `closed_book_worker.sh`, commit `cbe4ed0`). Offline vLLM (TP=8) samples $n=4$ tool-free answers per question (temp 1.0, top-p 0.95, 160 max tokens, short-answer system prompt) and scores sub-EM against the alias set; a question answerable from memory is dropped. Throughput ≈ 14.6 k tok/s (~ 7 min per split on one 8×H100 worker).

split	total	parametric $\geq 1/4$	parametric $\geq 2/4$	keep($< 2/4$)	pass-hist 0 / 1 / 2 / 3 / 4
base	35,583	28.8%	24.0%	27,059	25324 / 1735 / 1095 / 1077 / 6352
lrm	35,054	10.4%	7.9%	32,275	(LRM is far less parametric)

The base histogram is strongly bimodal (the model either knows a fact 4/4 or not at all), so the $\geq 2/4$ threshold separates cleanly. `keep_ge2.jsonl` (primary) and strict `keep_ge1.jsonl` are written per split; after also dropping the invalid slice the base diag set is 22,418 questions.

Stage 2 — in-setup pass-rate filter (commit `bca746b`; running at time of writing). Each kept question gets **8 independent in-setup rollouts** (search tool, wiki-18, 8 turns, temp 1.0) under the base policy, via the trainer’s `val_only` mode. Two implementation details matter:

- *Replication in data, not sampling:* the val env pool is sized `env_num = data.val_batch_size` (group_n = 1), and `SearchMultiProcessEnv.reset` rejects batches larger than the pool, so `val_kwargs.n=8` would crash. Instead `make_diag_parquet.py` writes each question 8× adjacent into the val parquet and keeps `val_kwargs.n=1`, `do_sample`, `T=1.0`.

- *Scoring* (`passrate_from_dump.py`): per-turn rollout dumps (`DUMP_TRAIN_SAMPLE=1`) are grouped by `traj_uid`; the question is parsed from the turn-0 observation; a trajectory passes if `env_won` at the terminal turn. Trajectories whose observations contain retriever-outage markers ("`Search error:`" / timeout text) are excluded; a question is scored only with ≥ 4 clean rollouts. The final keep-set is the **informative-gradient band** $0 < \text{pass} < n$ (drops all-fail and all-pass questions, which give zero GRPO gradient).

After stage 2, `band_keep.jsonl` feeds back through the converter to produce the filtered train/val parquets for the P2 relaunch (token-GRPO vs. turn-PPO, then GiGPO/HCAPO).

3 Context-length and episode configuration

Protocol configs/`protocol_asearcher_8turn_4b.sh` (commit 6d6aff8):

knob	value	rationale / evidence
<code>MAX_PROMPT_LENGTH</code>	16,384	fits 8 turns of history; P1 measured max seqlen 14,263 < 16,384 with 0% prompt clipping
<code>data.truncation</code>	left	oldest history dropped first if the cap is ever hit
<code>MAX_RESPONSE_LENGTH</code>	512/turn	base model clips $\sim 15\%$ at 512 (long think blocks); trained policy drops to 0.7–3% (P2 logs, resp-len p50 131–180 tok)
<code>MAX_STEPS</code> (turn budget)	8	rung-3 feasibility budget (paper uses 32–128 with a larger model + live web; our local-RAG/4B setting is deliberately smaller)
<code>HISTORY_LENGTH</code>	8	full-episode history in the prompt template
<code>TOPK</code>	3	docs returned per search
<code>SEARCH_DOC_MAX_WORDS</code>	180	per-doc guard in <code>_passages2string</code> ; never fires on wiki-18 (~ 103 -word chunks), protects against long-doc corpora
<code>enable_thinking</code>	False	Qwen3-4B-Instruct-2507 is non-thinking by construction

Memory headroom at these lengths ($8 \times \text{H100-80GB}$, FSDP + co-located vLLM at `gpu_memory_utilization=0.6`, gradient checkpointing, `MICRO_BSZ=1`): P1/P2 peak **53–70 GB/GPU** with no OOM at the full 1,280-trajectory batch; GPU-faiss adds ~ 7.6 GB/GPU (61 GB flat index sharded across 8 GPUs).

4 Training infrastructure

Hardware and launch. One attached Merlin/Arnold worker ($8 \times \text{H100-SXM-80GB}$, `mlsys_inference` queue) per condition, launched with `setuid mlx worker launch ...` (client lifetime = script lifetime; workers do not self-terminate — kill via `mlx worker kill`). Each fresh pod re-stages the 61 GB e5 index + 14 GB wiki-18 corpus HDFS→/tmp (~ 15 –30 min); the training venv and conda retriever env persist on shared /home.

Environments. Training venv `envs/agentric-rl-ca`: Python 3.11, vLLM 0.11.0, torch 2.8.0+cu128, flash-attn 2.8.3.post1, verl vendored in-repo. Retriever conda env `envs/searchr1-retr-conda` (micromamba, conda-forge): `python=3.11, mkl=2024.*, faiss-gpu=1.10.0, pytorch==cpu*, transformers, fastapi, uvicorn`. The `mkl=2024.*` **pin is load-bearing**: faiss 1.10 cuda129_mkl links `libmkl_intel_lp64.so.2`; mkl 2026 ships `.so.3` (ImportError). e5 query encoding runs on CPU torch so faiss-gpu owns the CUDA context (commit feb047c).

Retriever serving. In-repo `examples/search/retriever/retrieval_server.py` with `--faiss_gpu`, co-located on the training worker; batch HTTP contract `{"queries": [...]}` → `{"result": [[{document, score}]]}` (Search-R1 lineage; schema fix d03e302).

Trainer configuration (`run_condition.sh` + protocol): train batch $256 \times \text{group } 5 = 1,280$ trajs/step; `PPO_MINI_BATCH` 512 (locked; `MICRO_BSZ` override only changes gradient-accumulation chunking); LR $1e-6$; KL coef 0.001 (low-var-KL in loss for GRPO); $\gamma=1$; 150 steps; val 256 every 25 steps; wandb project `ca-rung3-4b-feasibility`. Observed ~ 442 s/step (generation-dominated at 16k). `HF_DATASETS_CACHE=/tmp` on every worker (the corpus Arrow cache once filled the shared 125 GB /home; commit 89415c8).

4.1 Reliability fixes from the stage-2 campaign (2026-07-28)

Six worker attempts exposed four distinct infra failure modes; all fixes are in `diag_worker4.sh` + wrappers and apply to future long rollout jobs:

1. **Pod preflight** (wrappers): some pods arrive without `libnvidia-ml.so` (CUDA works, but Ray’s `pynvml` autodetect sees 0 GPUs and dies 10 min in) or with a dead HDFS FUSE mount. Both are probed with a 3-min retry, then fail-fast so a relaunch draws a fresh pod.
2. **Retriever watchdog**: health-probe every 2 min; on 3 consecutive failures the server is restarted. Restart must use **SIGKILL** + **wait-for-dead**: `SIGTERM` leaves `uvicorn` “waiting for connections to close” forever on hung requests — the stuck instance holds its GPU shards, the port, and its `micromamba` parent.
3. **No micromamba in the restart path**: `micromamba`’s lock dir lives on shared /home (`~/.cache/mamba/proc`) and deadlocks across pods/stuck processes; the watchdog launches `$RETR_ENV/bin/python` directly.

4. **BLAS/OMP thread caps** (OMP/OPENBLAS/MKL_NUM_THREADS=8): the root cause of the recurring silent hangs — e5 CPU encoding at default OMP (=188 cores) under uvicorn’s 40-thread pool livelocks the server. With the caps, the retriever ran hang-free at 1,280-trajectory concurrency (vs. restarts every 10–60 min without). Rollout concurrency for the diagnostic is VAL_BATCH_OVERRIDE=1280 (P2-proven; 2,048 aggravated retriever load). Dumps are written to worker-local /tmp (tens of GB), never shared /home; only pass-rate summaries + zstd-compressed dumps go to HDFS.

5 Reproduce / artifacts

- Scripts: scripts/asearcher/{convert_base_to_parquet, closed_book_filter, make_diag_parquet, passrate_from_dump}.p, workers closed_book_worker.sh, diag_worker4.sh, wrappers diag_wrap_{base,lrn}.sh, launcher scripts/run_condition.sh, protocol configs/protocol_asearcher_8turn_4b.sh.
- Data (HDFS .../users/xiaoxuan/agentica_rl_ca/): data_asearcher_raw/, data_asearcher_base/ (unfiltered v1), logs/closed_book_filter/{base,lrn}/ (stats + keep-sets), logs/passrate_diag_{base,lrn}/ (band keep-sets, in flight).
- Key commits: 6d6aff8 (data + cap + protocol), cbe4ed0 (closed-book), bca746b (stage-2 diag), feb047c/8ae3f16 (conda retriever), d03e302 (schema), 89415c8 (HF cache), plus the 2026-07-28 hardening series through diag_worker4.